

Git, GitHub, GitLab & Bitbucket

The Complete End-to-End Guide for Java Full Stack Developers

Branching Strategies • Cherry-Pick • Rebase • Merge Conflicts • PR vs MR • CI/CD
• Organization Governance

With Real-Time Spring Boot + React Scenarios, Flow Diagrams and Interview Q&A

A practical, simple-language reference for interview preparation and daily real-world use

Table of Contents

1. Before Git vs After Git — Why Version Control Exists
2. Git vs GitHub vs GitLab vs Bitbucket
3. Git Architecture — Working Directory, Staging, Local & Remote Repo
4. Installing Git & First-Time Setup
5. Real-Time Project Setup — Spring Boot (Backend) + React (Frontend)
6. Core Git Commands with Real Examples
7. Understanding Branches
8. Git Branching Strategies Used in Real Companies
9. Merge vs Rebase — Deep Dive with Diagrams
10. Git Cherry-Pick — Why and When (Real Hotfix Scenario)
11. Git Stash — Saving Work in Progress
12. Reset vs Revert vs Checkout vs Restore
13. Handling Merge Conflicts Step by Step (Java + React Example)
14. Tags & Release Management
15. Git Hooks — Automating Quality Checks
16. .gitignore for Java Full Stack Projects
17. Working with Remotes — Fetch vs Pull, Forking
18. Pull Request vs Merge Request — Who Approves & How
19. CI/CD Integration with Git (GitHub Actions / GitLab CI)
20. Organization-Level Git Governance (Top-Down)
21. A Real Week in the Life of a Java Full Stack Developer
22. Common Git Issues & How to Fix Them
23. Git Best Practices for Full Stack Teams
24. Git Command Cheat Sheet
25. Interview Questions & Answers (Basic → Advanced → Scenario Based)
26. Final Summary

1. Before Git vs After Git

1.1 Life Before Version Control

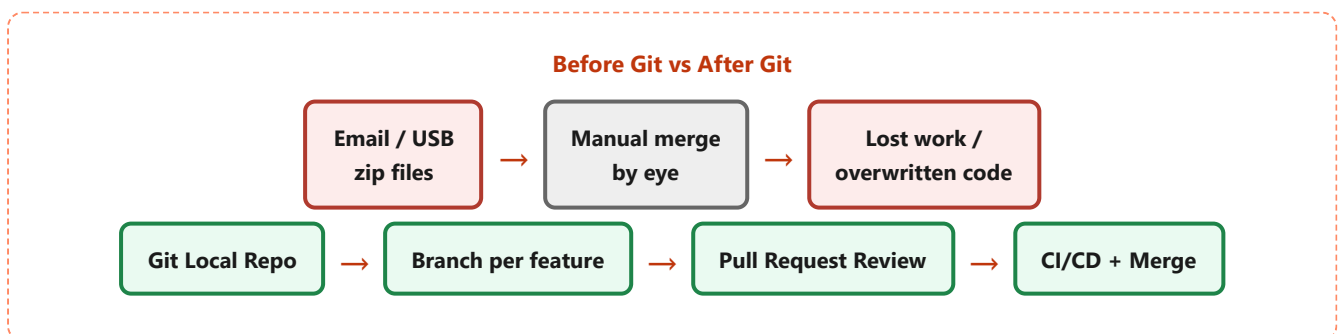
Imagine a Java full stack team of 6 developers building an e-commerce app (Spring Boot backend + React frontend) without Git:

- Developers email each other zip files like `OrderService_final_v2_fixed.zip` , `OrderService_final_FINAL.zip` .
- Two developers edit `OrderController.java` at the same time; one overwrites the other's work.
- There is no history — nobody knows who changed what, or why a bug was introduced.
- To go back to yesterday's working version, someone must remember to keep a manual backup.
- Deploying to production means manually copying files to a server — very risky and error prone.
- Code review is done by reading printed code or screen-sharing — no structured process.

Real pain: A developer accidentally deletes a working `application.yml` config a day before release, and there is no way to recover it. The whole team stays late to rebuild it from memory.

1.2 Life After Git

- Every change is tracked with an author, timestamp, and message — full history forever.
- Multiple developers work on the same files in parallel using **branches**, without blocking each other.
- Any file or the entire project can be restored to any previous point in seconds.
- Code changes go through **Pull Requests / Merge Requests** with mandatory review and automated tests before reaching production.
- Git works completely offline (it is distributed) — you can commit, view history, and branch without internet; you only need internet to `push` / `pull` to share with the team.
- Combined with GitHub/GitLab/Bitbucket, it becomes a full collaboration and deployment platform: issues, CI/CD pipelines, code owners, security scanning, and release management.



1.3 What Git Actually Is

Git is a **Distributed Version Control System (DVCS)** created by Linus Torvalds in 2005. "Distributed" means every developer has a full copy of the entire project history on their own laptop — not just the latest files.

2. Git vs GitHub vs GitLab vs Bitbucket

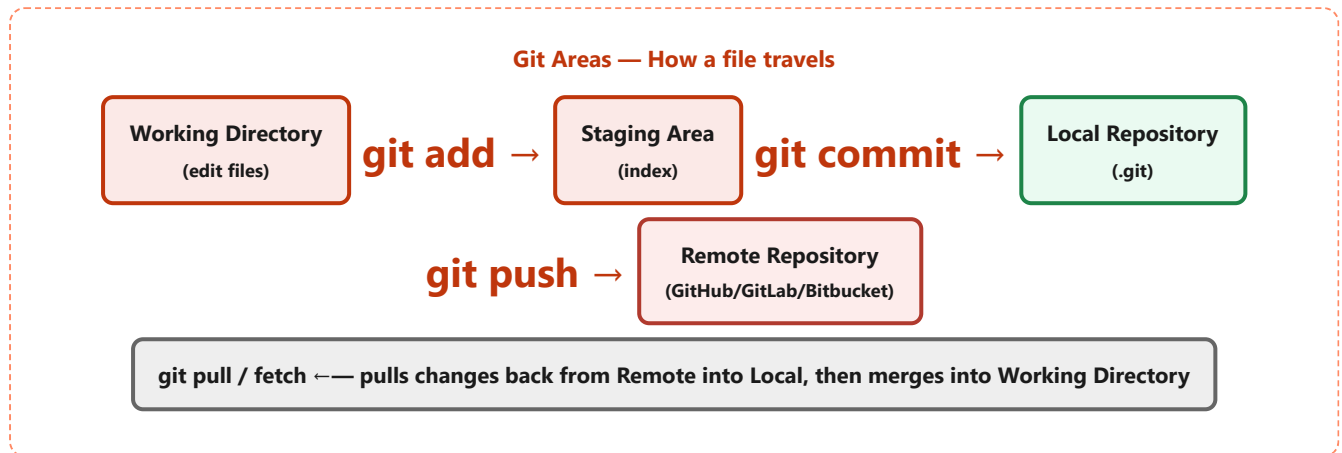
Simple analogy: Git is the engine (the version control tool itself, runs on your machine). GitHub, GitLab, and Bitbucket are cloud garages/showrooms that host your Git repositories online and add collaboration features like Pull Requests, CI/CD, issue tracking, and access control.

Feature	Git	GitHub	GitLab	Bitbucket
What it is	Version control tool (CLI)	Hosting platform + collaboration	Hosting platform + built-in DevOps	Hosting platform (Atlassian ecosystem)
Owner	Open source (Linux Foundation)	Microsoft	GitLab Inc.	Atlassian
Review terminology	—	Pull Request (PR)	Merge Request (MR)	Pull Request (PR)
Built-in CI/CD	—	GitHub Actions	GitLab CI/CD (very mature, built-in)	Bitbucket Pipelines
Issue tracking	—	GitHub Issues	GitLab Issues	Jira (tight integration)
Best known for	—	Open source community, largest ecosystem	Single DevOps platform (plan→deploy→monitor)	Enterprise teams already using Jira/Confluence
Typical company use	Everywhere	Startups, product companies, OSS	Enterprises wanting one integrated tool	Enterprises using Atlassian suite

Real time: In many Java full stack teams you will see: code in GitHub/GitLab/Bitbucket, tickets in Jira, CI/CD pipeline auto-triggered on every push, and Slack/Teams notification on PR/MR events.

3. Git Architecture

Git has four conceptual areas a file moves through:



3.1 Real Example

```
# You edit OrderController.java in your IDE → Working Directory
git add OrderController.java                # → Staging Area
git commit -m "Add validation to order API" # → Local Repository (.git)
git push origin feature/order-validation    # → Remote Repository (GitHub)
```

Each commit is identified by a unique SHA-1 hash (e.g., `a3f5e21`). Git stores commits as a linked chain — each commit points to its parent, forming the project history graph.

4. Installing Git & First-Time Setup

```
# Windows: download from git-scm.com, or
winget install --id Git.Git -e

# Verify
git --version

# One-time global identity (used in every commit you make)
git config --global user.name "Priya Sharma"
git config --global user.email "priya.sharma@company.com"

# Recommended defaults
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
git config --global pull.rebase false
```

5. Real-Time Project Setup — Java Full Stack

Typical repo layout for a Spring Boot + React project (can be one mono-repo or two separate repos):

```
ecommerce-app/  
├── backend/           # Spring Boot (Java, Maven/Gradle)  
│   ├── src/main/java/com/company/order/  
│   ├── pom.xml  
│   └── application.yml  
├── frontend/         # React  
│   ├── src/  
│   ├── package.json  
│   └── .env  
├── .gitignore  
└── README.md
```

```
git init  
git remote add origin https://github.com/company/ecommerce-app.git  
git add .  
git commit -m "Initial project skeleton: Spring Boot + React"  
git branch -M main  
git push -u origin main
```

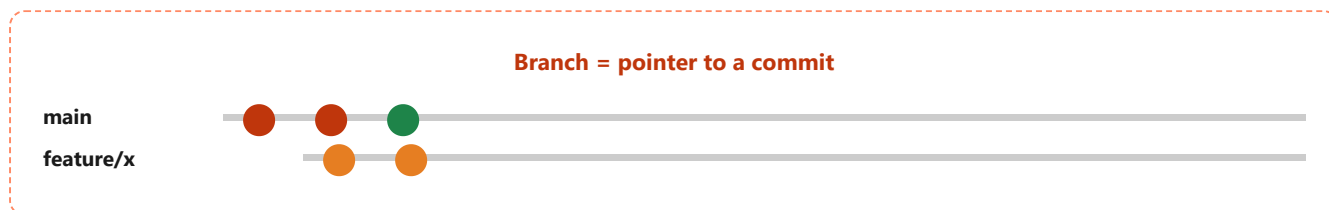
Real time note: Larger companies (Amazon, banks, etc.) often keep backend and frontend in **separate repositories** with independent pipelines, versioning and deployments. Startups often prefer a single **mono-repo** for simplicity.

6. Core Git Commands with Real Examples

Command	Purpose	Real example
<code>git clone <url></code>	Copy a remote repo to your machine	<code>git clone https://github.com/company/ecommerce-app.git</code>
<code>git status</code>	See changed/staged/untracked files	Before every commit
<code>git add <file></code>	Move change to staging area	<code>git add OrderService.java</code>
<code>git commit -m "msg"</code>	Save a snapshot with message	<code>git commit -m "JIRA-231: fix null pointer in order total"</code>
<code>git push</code>	Upload local commits to remote	<code>git push origin feature/order-total-fix</code>
<code>git pull</code>	fetch + merge remote changes	<code>git pull origin develop</code>
<code>git fetch</code>	Download remote changes without merging	used before rebase
<code>git log --oneline --graph</code>	View commit history visually	Debugging release history
<code>git diff</code>	Show unstaged changes	Reviewing your own change before commit
<code>git branch</code>	List / create branches	<code>git branch feature/payment-gateway</code>
<code>git checkout -b <name></code> / <code>git switch -c <name></code>	Create + move to new branch	<code>git switch -c feature/JIRA-231-order-fix</code>
<code>git merge <branch></code>	Combine another branch into current	<code>git merge feature/order-fix</code>
<code>git rebase <branch></code>	Replay commits on top of another branch	<code>git rebase develop</code>
<code>git cherry-pick <hash></code>	Apply one specific commit elsewhere	<code>git cherry-pick 9af31bc</code>
<code>git stash</code>	Temporarily shelve uncommitted work	Switching branch urgently for a hotfix
<code>git reset</code>	Move HEAD/undo commits	<code>git reset --soft HEAD~1</code>
<code>git revert <hash></code>	Undo a commit safely with a new commit	Undo a bad commit already pushed to main
<code>git tag</code>	Mark a release point	<code>git tag -a v1.2.0 -m "Release 1.2.0"</code>

7. Understanding Branches

A branch is simply a movable pointer to a commit. Creating a branch is instant and cheap — this is what makes Git branching-friendly compared to older tools like SVN.



Real example: a developer picks up JIRA ticket **JIRA-231** to fix an order total bug.

```
git checkout develop
git pull origin develop
git switch -c feature/JIRA-231-order-total-fix
# ... edit OrderService.java, OrderServiceTest.java ...
git add .
git commit -m "JIRA-231: fix order total rounding bug"
git push -u origin feature/JIRA-231-order-total-fix
# then open a Pull Request / Merge Request into develop
```

8. Git Branching Strategies Used in Real Companies

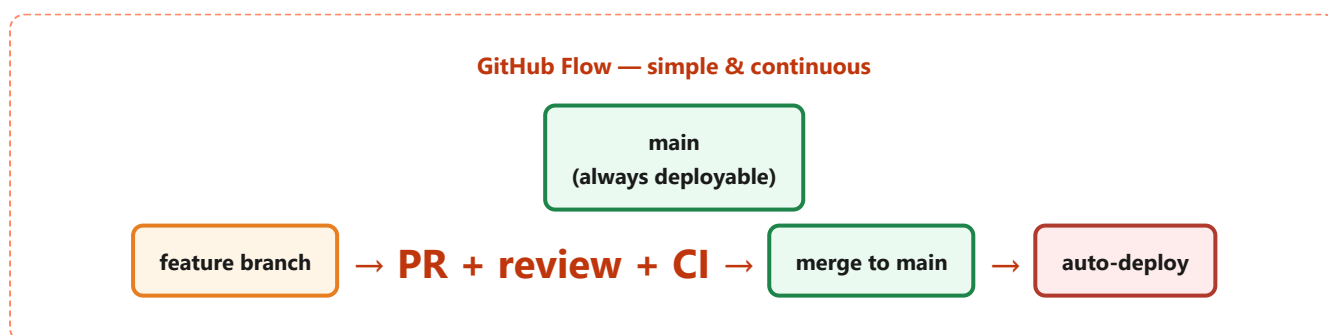
8.1 Git Flow (used in regulated / release-based products, e.g., banking, insurance)



- **main** — always production-ready, tagged with version numbers.
- **develop** — integration branch where finished features accumulate.
- **feature/*** — one branch per ticket/feature, branched from develop, merged back to develop.
- **release/*** — cut from develop when preparing a release, only bug fixes allowed, merged into both main and develop.
- **hotfix/*** — cut from main to fix urgent production bugs, merged into both main and develop.

Real scenario: Production has a critical bug in payment rounding on a Friday evening. Team creates **hotfix/payment-rounding** from **main**, fixes it, tests it, merges to **main** (deployed immediately) AND to **develop** (so the fix isn't lost in the next release).

8.2 GitHub Flow (used in fast-moving SaaS/product teams with continuous deployment)



Only one long-lived branch: **main**. Every change is a short-lived feature branch merged via PR after review + passing CI, then deployed immediately (or automatically).

8.3 GitLab Flow (middle ground — environment branches)

Adds environment branches on top of GitHub Flow: **main** → **pre-production** → **production**, so each merge is promoted stage by stage instead of deploying straight to prod.

8.4 Trunk-Based Development (used by Google, Meta-scale teams, high maturity CI/CD)

- Everyone commits to `main` (trunk) directly or via very short-lived branches (a few hours).
- Unfinished features are hidden behind **feature flags** instead of long branches.
- Requires strong automated testing and CI, since main is always deploying.

8.5 Which strategy for which real-world team?

Team type	Recommended strategy	Why
Banking/insurance, scheduled releases	Git Flow	Needs release branches, strict QA sign-off, hotfix path
SaaS product, deploys many times a day	GitHub Flow	Simple, fast, PR + CI gate before merge
Enterprise with staged environments	GitLab Flow	Clear promotion across dev → staging → prod
Large-scale, mature CI/CD, feature flags	Trunk-Based	Avoids long-lived branch merge pain at scale

8.6 Example: Real Java Full Stack Sprint Using Git Flow

```
# 1. Start feature from develop
git checkout develop && git pull
git switch -c feature/JIRA-410-add-cart-api

# 2. Work on backend (Spring Boot) + frontend (React) in same branch
git add backend/src/main/java/com/company/cart/CartController.java
git add frontend/src/components/Cart.jsx
git commit -m "JIRA-410: add cart REST API and Cart UI"

# 3. Push and open Pull/Merge Request into develop
git push -u origin feature/JIRA-410-add-cart-api

# 4. After 2 approvals + CI green -> squash merge into develop

# 5. Release day: cut release branch
git checkout develop && git pull
git switch -c release/1.4.0

# 6. QA finds a bug on release branch, fix directly there
git commit -m "fix: cart total tax calculation for release 1.4.0"

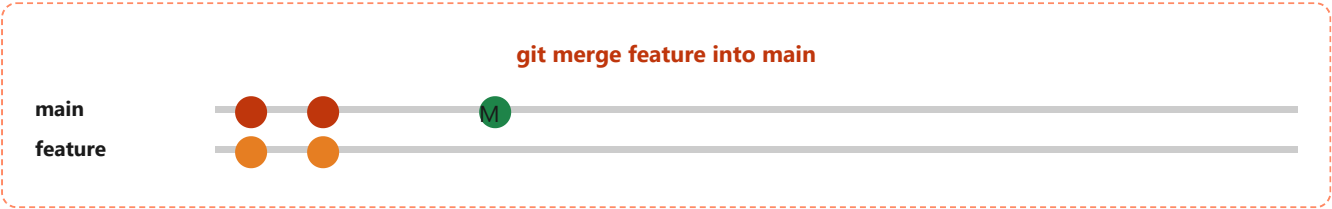
# 7. Release approved -> merge to main and tag
git checkout main && git merge --no-ff release/1.4.0
git tag -a v1.4.0 -m "Release 1.4.0"
git push origin main --tags

# 8. Merge back to develop so the fix is not lost
git checkout develop && git merge --no-ff release/1.4.0
```

9. Merge vs Rebase — Deep Dive

9.1 Merge

Combines two branch histories and creates a new "merge commit" with two parents. History is preserved exactly as it happened.



```
git checkout main
git merge feature/JIRA-410-add-cart-api
# Creates a merge commit; both histories are kept, non-destructive
```

9.2 Rebase

Moves (replays) your branch commits on top of another branch's latest commit, producing a straight, linear history — as if you started your work after the latest change.



```
git checkout feature/JIRA-410-add-cart-api
git fetch origin
git rebase origin/develop
# resolve conflicts if any, then:
git rebase --continue
git push --force-with-lease origin feature/JIRA-410-add-cart-api
```

9.3 When to use which (real team rules)

Situation	Use	Reason
Merging a finished feature branch into develop/main via PR	Merge (often squash merge)	Keeps a clear "feature merged" point, safe for shared branches
Updating your own feature branch with latest develop before opening a PR	Rebase	Clean linear history, easy to review, no noisy merge commits
Branch already pushed and others are using it	Merge, not rebase	Rebase rewrites history — dangerous for shared/public branches
Need very clean history for open-source project	Rebase + squash	One clean commit per feature

Golden Rule: Never rebase a branch that other people are also working on / have already pulled — it rewrites commit hashes and breaks their history.

10. Git Cherry-Pick — Why and When

`git cherry-pick` takes one specific commit from one branch and applies it onto another branch, without merging the entire branch.



Why we use it (real reasons)

- A critical hotfix commit exists on `release/1.4` but must also go to `main` immediately, without pulling in unfinished release-branch changes.
- You accidentally committed a small fix to the wrong branch and need it on the correct one.
- Backporting a bug fix to an older supported version branch (e.g., `release/1.3` still used by a client).
- Picking a single useful commit from a colleague's experimental/abandoned branch.

Real scenario (Java full stack)

QA finds a critical `NullPointerException` in `PaymentService.java` already fixed on `release/1.4` (commit `9af31bc`), but `main` is already live in production and cannot take the whole unfinished release branch.

```
git log release/1.4 --oneline      # find the commit hash, e.g. 9af31bc
git checkout main
git pull origin main
git cherry-pick 9af31bc
# resolve conflicts if any
git push origin main
```

If cherry-picking a merge commit, use `git cherry-pick -m 1 <hash>` to tell Git which parent line to follow.

11. Git Stash — Saving Work in Progress

Stash temporarily shelves uncommitted changes so you can switch branches cleanly, then bring them back later.

Real scenario

You are mid-way through editing `InventoryService.java` for a new feature when your manager asks you to urgently fix a production bug on `main`.

```
git stash push -m "WIP: inventory reorder feature"
git checkout main
git pull
git switch -c hotfix/stock-count-bug
# fix, commit, push, PR ...
git checkout feature/inventory-reorder
git stash list
git stash pop      # restores your WIP changes
```

12. Reset vs Revert vs Checkout vs Restore

Command	What it does	Safe on shared/pushed branch?	Real example
<code>git reset --soft HEAD~1</code>	Undo last commit, keep changes staged	No (rewrites history)	Combine last 2 commits before pushing
<code>git reset --mixed HEAD~1</code>	Undo commit, keep changes unstaged	No	Redo commit message/content
<code>git reset --hard HEAD~1</code>	Completely discard last commit and changes	No — destructive	Throw away a bad local experiment
<code>git revert <hash></code>	Create a NEW commit that undoes an old commit	Yes — safe, preferred for shared branches	Undo a bad commit already merged to main in production
<code>git checkout <file></code>	Discard local changes in a file (old syntax)	Local only	Discard accidental edit
<code>git restore <file></code>	Modern replacement for discarding file changes	Local only	<code>git restore application.yml</code>

Real rule used in every company: Never `reset --hard` or force-push on `main` / `develop`. To undo something already pushed and shared, always use `git revert`.

13. Handling Merge Conflicts — Step by Step

A conflict happens when Git cannot automatically decide how to combine changes because two branches edited the same lines differently.

Real example: two developers edit application.yml

```
<<<<<< HEAD
server:
  port: 8080
=====
server:
  port: 9090
>>>>>> feature/JIRA-512-new-port
```

Resolution steps

1. `git status` — see which files are in conflict.
2. Open each conflicted file, find the `<<<<<<` / `=====` / `>>>>>>` markers.
3. Decide the correct final content (talk to the other developer if unsure), delete the markers.
4. `git add <file>` to mark it resolved.
5. `git commit` (if merging) or `git rebase --continue` (if rebasing).
6. Run tests locally before pushing.

Also common in Java projects: conflicts in `pom.xml` dependency versions, and in React projects: conflicts in `package-lock.json` — for lock files, usually safest to delete and regenerate with `npm install` after resolving `package.json`.

14. Tags & Release Management

Tags mark a specific commit as a release point, typically using Semantic Versioning: `MAJOR.MINOR.PATCH` (e.g., `v2.3.1`).

```
git tag -a v2.3.0 -m "Release 2.3.0 - Add cart and checkout APIs"
git push origin v2.3.0
git tag                # list all tags
git checkout v2.3.0    # inspect exact code that was released
```

In GitHub/GitLab, tags are used to auto-trigger release pipelines that build a Docker image, e.g. `myapp:2.3.0`, and deploy it to production.

15. Git Hooks — Automating Quality Checks

Hooks are scripts that run automatically at certain Git events, stored in `.git/hooks/` (or shared via tools like Husky for Node/React projects).

Hook	Triggered	Real Java full stack use
pre-commit	Before a commit is created	Run Checkstyle/Spotless on Java, ESLint on React code
commit-msg	After message is written	Enforce conventional commit format: <code>feat:</code> , <code>fix:</code> , <code>JIRA-123:</code>
pre-push	Before push to remote	Run unit tests (<code>mvn test</code> / <code>npm test</code>) — block push if failing

```
#!/bin/sh (.git/hooks/pre-commit)
mvn -q -f backend/pom.xml checkstyle:check
cd frontend && npx eslint src/ --max-warnings=0
```

16. .gitignore for Java Full Stack Projects

```
# --- Java / Maven / Gradle (backend) ---
target/
build/
*.class
.mvn/
*.iml
.idea/

# --- Node / React (frontend) ---
node_modules/
build/
dist/
.env
.env.local

# --- OS / IDE ---
.DS_Store
.vscode/
*.log
```

Never commit secrets: `.env` , `application-secrets.yml` , API keys, DB passwords. Use a secrets manager (Vault, AWS Secrets Manager) or CI/CD environment variables instead.

17. Working with Remotes — Fetch vs Pull, Forking

Command	Behavior
<code>git fetch</code>	Downloads new commits from remote but does NOT touch your working files/branch
<code>git pull</code>	<code>git fetch</code> + <code>git merge</code> (or rebase, if configured) — updates your current branch immediately

Real time habit: Senior developers often prefer `git fetch` then review incoming changes (`git log origin/main`) before merging, especially on critical branches.

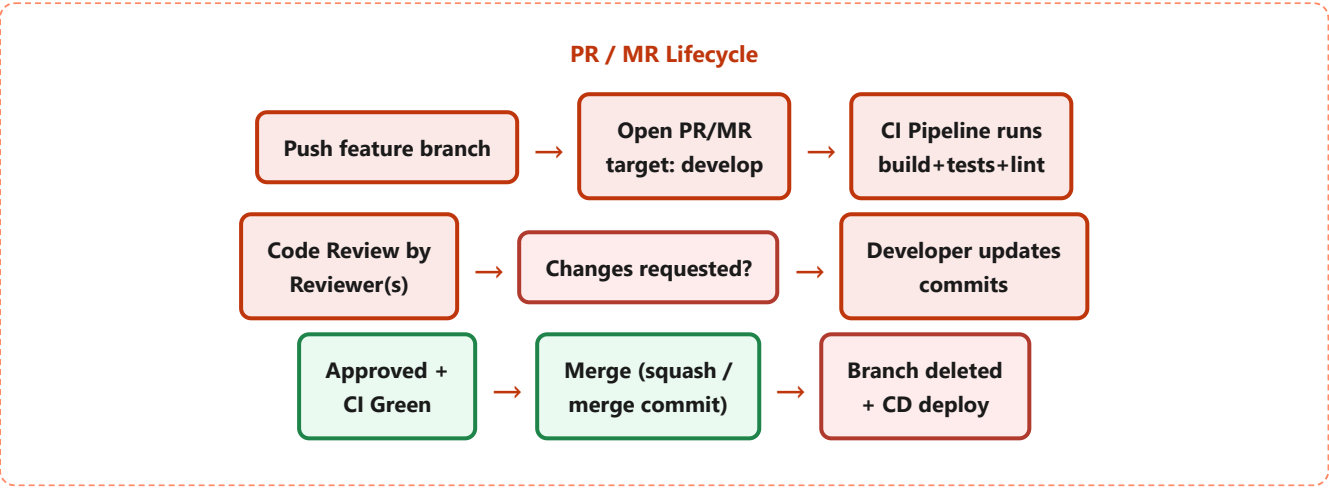
Forking workflow (common in open source and some enterprises)

1. Fork the company/OSS repo into your own account.
2. Clone your fork, add the original as `upstream` : `git remote add upstream https://github.com/company/repo.git`
3. Create a feature branch, commit, push to your fork.
4. Open a Pull Request from your fork's branch into the original repo.
5. Periodically sync: `git fetch upstream && git rebase upstream/main`

18. Pull Request vs Merge Request — Who Approves & How

18.1 Are they different?

Functionally they are the same concept with different names: a request to merge your branch into a target branch, after review. **GitHub and Bitbucket** call it a **Pull Request (PR)**. **GitLab** calls it a **Merge Request (MR)**.



18.2 Who approves?

Role	Responsibility
Code Owner / Tech Lead	Mandatory approver for critical modules (defined in <code>CODEOWNERS</code> file)
Senior Developer / Peer Reviewer	Reviews code quality, logic, tests, naming, design patterns
QA / Automated CI	Must pass build, unit tests, code coverage, security scan before merge is allowed
Repository Maintainer/Admin	Can override or merge in emergencies (used rarely, tracked)

Enforced via **Branch Protection Rules** (GitHub) / **Protected Branches** (GitLab, Bitbucket):

- Require at least N approvals (commonly 1–2) before merge is allowed.
- Require status checks (CI build/tests) to pass.
- Require CODEOWNERS approval for specific critical folders (e.g., `/backend/payment/**`).
- Disallow direct pushes and force-pushes to `main` / `develop`.
- Require branches to be up to date before merging.

```
# CODEOWNERS file example
/backend/src/main/java/com/company/payment/ @priya-lead @finance-team
/frontend/src/components/checkout/ @rahul-frontend-lead
* @default-reviewers-team
```

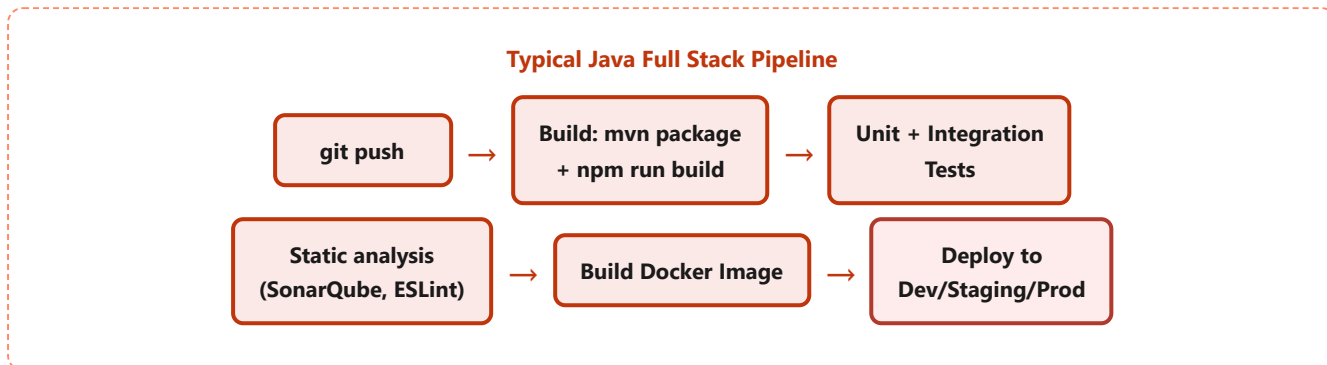
18.3 Real Scenario

A developer opens a PR titled `JIRA-410: Add cart API` targeting `develop`. GitHub automatically requests review from the CODEOWNERS of `/backend/cart/`. GitHub Actions runs `mvn test` and `npm test`. One reviewer requests a change ("add

null check for empty cart"). Developer pushes a fix commit. CI reruns, passes. Reviewer approves. Branch protection requires 2 approvals + green CI — once satisfied, the "Squash and merge" button becomes enabled.

19. CI/CD Integration with Git

CI/CD pipelines are triggered automatically by Git events (push, PR/MR open, tag creation).



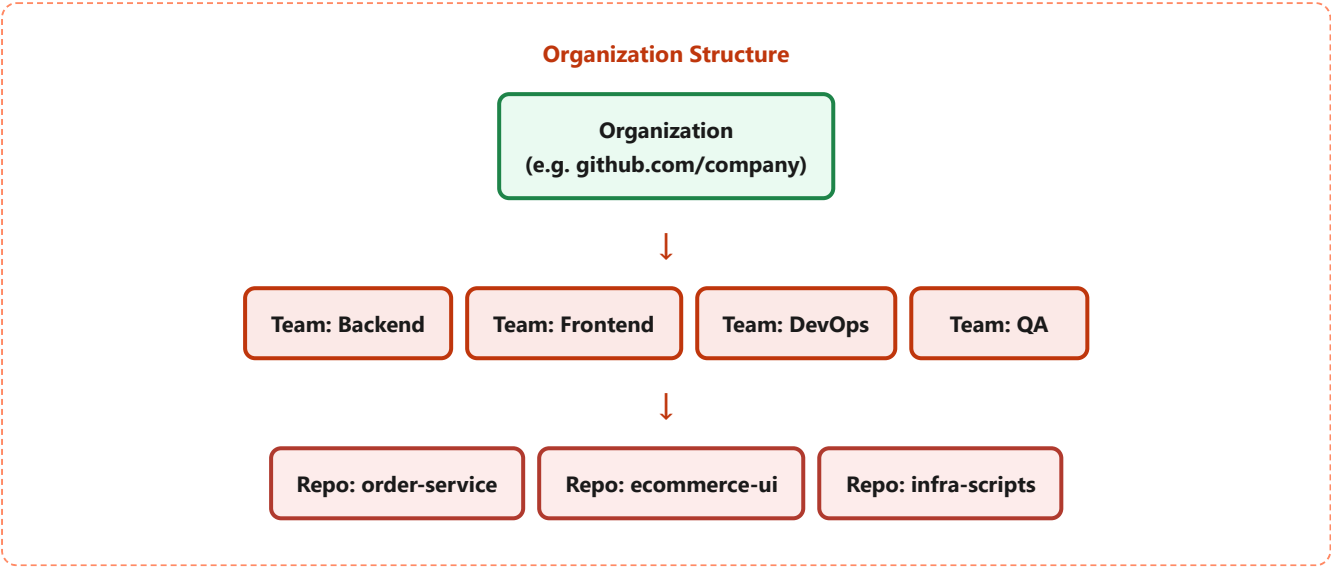
Example: GitHub Actions (backend)

```
name: backend-ci
on:
  pull_request:
    branches: [ develop, main ]
jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '17', distribution: 'temurin' }
      - run: mvn -f backend/pom.xml clean verify
```

Example: GitLab CI (frontend)

```
stages: [build, test, deploy]
build_frontend:
  stage: build
  script:
    - cd frontend && npm ci && npm run build
test_frontend:
  stage: test
  script:
    - cd frontend && npm test -- --watchAll=false
```

20. Organization-Level Git Governance (Top-Down)



20.1 Roles & Permission Levels

Role	GitHub term	GitLab term	Typical rights
Highest admin	Owner	Owner	Manage billing, members, delete org/repos, all settings
Repo admin	Admin (repo role)	Maintainer	Manage branch protection, merge without restriction, manage settings
Contributor	Write	Developer	Push branches, open PR/MR, cannot change repo settings
Reviewer only	Triage	Reporter	Comment, review, manage issues, no code push
Read-only	Read	Guest	View code only

20.2 Governance Controls Used by Real Organizations

- **Branch protection rules** on `main` / `develop` : required reviews, required CI checks, no force-push.
- **CODEOWNERS** to force domain experts to review critical code (payment, auth, security modules).
- **SSO / SAML** login enforced at organization level, sometimes with IP allow-listing.
- **Audit logs** — who pushed, who approved, who changed settings, retained for compliance (banking/healthcare).
- **Repository visibility rules** — private by default; public repos require an approval process.
- **Secret scanning & dependency scanning** enabled org-wide (e.g., GitHub Advanced Security, GitLab Ultimate).
- **Merge strategy locked** at repo/org level (e.g., force "Squash and merge" only, to keep history clean).

21. A Real Week in the Life of a Java Full Stack Developer

Day	Activity	Git commands used
Monday	Pick JIRA-512, create feature branch, build backend endpoint	<code>git pull</code> , <code>git switch -c feature/JIRA-512</code> , <code>git add</code> , <code>git commit</code>
Tuesday	Build matching React component, test integration locally	<code>git commit</code> , <code>git push</code>
Wednesday	Rebase onto latest develop before opening PR; resolve conflict in <code>pom.xml</code>	<code>git fetch</code> , <code>git rebase origin/develop</code> , <code>git add</code> , <code>git rebase --continue</code>
Wednesday PM	Open PR, CI runs, reviewer requests changes	<code>git commit --amend</code> or new commit, <code>git push --force-with-lease</code>
Thursday	Urgent prod bug reported — stash current work, hotfix	<code>git stash</code> , <code>git switch main</code> , <code>git switch -c hotfix/...</code> , <code>git cherry-pick fix into develop</code>
Thursday PM	Back to feature work	<code>git switch feature/JIRA-512</code> , <code>git stash pop</code>
Friday	PR approved, squash merged, release cut and tagged, deployed via CI/CD	<code>git tag -a v2.4.0</code> , <code>git push --tags</code>

22. Common Git Issues & How to Fix Them

Issue	Cause	Fix
"Detached HEAD" state	Checked out a commit/tag directly instead of a branch	<code>git switch -c temp-fix</code> to save work, or <code>git switch main</code> to go back
Merge conflict on pull	Same lines changed both locally and remotely	Resolve markers manually, <code>git add</code> , <code>git commit</code>
Accidentally committed to <code>main</code>	Forgot to create a feature branch	<code>git branch feature/oops</code> then <code>git reset --hard origin/main</code> on main
"Updates were rejected because the remote contains work you do not have"	Remote has new commits you don't have locally	<code>git pull --rebase</code> then push again
Push rejected — large file / secret detected	Committed a large binary or a credential file	Remove file with <code>git rm --cached</code> , use <code>git filter-repo</code> /BFG to purge history, add to <code>.gitignore</code>
"Permission denied (publickey)"	SSH key not set up or not added to GitHub/GitLab account	<code>ssh-keygen -t ed25519</code> , add public key to your Git host account, test with <code>ssh -T git@github.com</code>
Diverged branches (both ahead and behind)	Local and remote both have new commits	<code>git pull --rebase</code> (or merge), resolve conflicts, push
Accidentally deleted a branch with unmerged work	Ran <code>git branch -D</code> too early	<code>git reflog</code> to find the last commit hash, then <code>git checkout -b recovered-branch <hash></code>
Wrong commit message already pushed	Typo, wrong ticket number	If not shared yet: <code>git commit --amend</code> + <code>push --force-with-lease</code> ; if shared, leave it or add a follow-up commit
".gitignore not working" for already-tracked file	File was committed before being ignored	<code>git rm --cached <file></code> , commit, then it respects .gitignore going forward
"Your branch and 'origin/main' have diverged" after a force-push by a teammate	Someone rewrote shared branch history (amend/rebase) that others already pulled	Agree on force-push etiquette (only on personal feature branches); affected teammates run <code>git fetch</code> + <code>git reset --hard origin/<branch></code> after confirming no local-only work is lost
Cloning a large repo is extremely slow	Full history includes large binary files never cleaned up	Use <code>git clone --depth 1</code> for a shallow clone when full history isn't needed, or migrate large assets to Git LFS

23. Git Best Practices for Full Stack Teams

- One feature/bugfix = one branch, named consistently: `feature/JIRA-123-short-desc` , `hotfix/...`.
- Small, frequent commits with clear messages (conventional commits: `feat:` , `fix:` , `chore:` , `refactor:`).
- Always `git pull --rebase` or fetch+rebase before starting new work to avoid unnecessary merge commits.
- Never commit secrets, credentials, or environment files.
- Keep PRs/MRs small (< 400 lines changed where possible) — easier and faster to review.
- Write meaningful PR descriptions: what changed, why, how tested, screenshots for UI.
- Use draft PRs for work-in-progress visibility without triggering full review.
- Protect `main` / `develop` with required reviews and passing CI.
- Delete branches after merge to keep the repo clean.
- Tag every production release for traceability and easy rollback.

24. Git Command Cheat Sheet

Category	Commands
Setup	<code>git init</code> , <code>git clone</code> , <code>git config</code>
Basic workflow	<code>git status</code> , <code>git add</code> , <code>git commit</code> , <code>git push</code> , <code>git pull</code>
Branching	<code>git branch</code> , <code>git switch -c</code> , <code>git checkout -b</code> , <code>git merge</code> , <code>git rebase</code>
Inspect	<code>git log</code> , <code>git diff</code> , <code>git show</code> , <code>git blame</code> , <code>git reflog</code>
Undo	<code>git reset</code> , <code>git revert</code> , <code>git restore</code> , <code>git checkout --</code>
Advanced	<code>git cherry-pick</code> , <code>git stash</code> , <code>git bisect</code> , <code>git tag</code> , <code>git rebase -i</code>
Remote	<code>git remote -v</code> , <code>git fetch</code> , <code>git push -u</code> , <code>git push --force-with-lease</code>
Cleanup	<code>git branch -d/-D</code> , <code>git rm --cached</code> , <code>git gc</code>

25. Interview Questions & Answers

25.1 Basic Level

Q1. What is Git and why is it called "distributed"?

A: Git is a distributed version control system. Unlike centralized systems (SVN), every developer has a full copy of the repository including complete history on their local machine, so most operations (commit, branch, log, diff) work offline.

Q2. Difference between Git and GitHub?

A: Git is the version control tool itself. GitHub is a cloud platform that hosts Git repositories and adds collaboration features (Pull Requests, Issues, Actions/CI-CD, access control).

Q3. What is the difference between `git fetch` and `git pull` ?

A: `fetch` downloads remote commits but does not change your working branch. `pull` = `fetch` + `merge` (or rebase), immediately updating your current branch.

Q4. What is the staging area (index)?

A: An intermediate area where changes are placed with `git add` before being permanently saved with `git commit`. It lets you commit only part of your changes.

Q5. What does `.gitignore` do?

A: Tells Git which files/folders to never track (e.g., `target/`, `node_modules/`, `.env`), keeping the repo clean and secrets safe.

25.2 Intermediate Level

Q6. Difference between merge and rebase?

A: Merge combines two histories with a new merge commit, preserving exact history (non-destructive). Rebase replays your commits on top of another branch, producing linear history but rewriting commit hashes (should not be done on shared/pushed branches).

Q7. What is cherry-pick and when do you use it?

A: Applies one specific commit from another branch onto the current branch. Used for backporting a hotfix to a release branch or main without merging the whole source branch.

Real world: A fix merged into `develOp` is needed urgently in production; instead of merging all of develop (which has unfinished features), you cherry-pick just that one commit into `main`.

Q8. Difference between `git reset` and `git revert` ?

A: `reset` moves the branch pointer and can rewrite/discard history (dangerous on shared branches). `revert` creates a brand-new commit that undoes changes, safe to use on shared/production branches because history isn't rewritten.

Q9. What is a detached HEAD state?

A: Happens when you checkout a specific commit or tag instead of a branch. Any new commits made there aren't attached to a branch and can be lost unless you create a new branch from that point.

Q10. What is git stash used for?

A: Temporarily saves uncommitted changes so you can switch branches cleanly (e.g., to handle an urgent hotfix), then restore your work later with `git stash pop`.

Q11. What is a fast-forward merge?

A: When the target branch has no new commits since the feature branch was created, Git simply moves the pointer forward instead of creating a merge commit.

Q12. What is `git rebase -i` (interactive rebase) used for?

A: To clean up local commit history before pushing/opening a PR — squash multiple small commits into one, reorder commits, edit or drop commits, fix commit messages.

25.3 Advanced / Scenario-Based (Real World)

Q13. Your PR has 15 messy "WIP" commits. How do you clean it up before review?

A: Use `git rebase -i HEAD~15`, mark commits to `squash` / `fixup` into one, write one clean commit message, then `git push --force-with-lease` (safe force push, only if branch is yours alone).

Q14. Production is broken from a bad merge to main. What do you do?

A: Prefer `git revert <merge-commit-hash> -m 1` to safely create a new commit undoing the bad merge without rewriting shared history, then redeploy immediately. Avoid `reset --hard` + force-push on main.

Q15. Two developers changed the same lines in `OrderService.java` — how do you resolve on merge?

A: Git marks conflict with `<<<<<< / ===== / >>>>>>`. Discuss with the other developer, manually pick/combine the correct logic, remove markers, run tests, `git add`, then commit or continue the merge/rebase.

Q16. You need a fix from a colleague's still-unfinished branch, but not the whole branch. What do you do?

A: `git cherry-pick <commit-hash>` from their branch onto yours or into main, avoiding the unfinished parts.

Q17. What's the difference between Pull Request and Merge Request, and who typically approves them?

A: Same concept — GitHub/Bitbucket call it PR, GitLab calls it MR: a request to merge a branch after review. Approved by designated reviewers/code owners (per `CODEOWNERS`) plus required passing CI checks, enforced through branch protection rules; repo admins/maintainers can merge once conditions are met.

Q18. How do branch protection rules and CODEOWNERS work together at an organization level?

A: Branch protection defines the merge gate for a branch (min approvals, required CI checks, no force-push). CODEOWNERS maps specific paths (e.g. `/payment/**`) to required reviewers, so critical code always gets domain-expert review before it can merge.

Q19. You accidentally pushed an API key to a public repo. What now?

A: Immediately rotate/revoke the exposed key, remove it from history using `git filter-repo` or BFG Repo-Cleaner, force-push the cleaned history, and add the file to `.gitignore`. Rotating the credential is the most important step — history rewrite alone is not enough since it may already be cached/forked.

Q20. Difference between Git Flow and Trunk-Based Development — when would you pick each?

A: Git Flow uses long-lived develop/release/hotfix branches, good for scheduled releases (banking, enterprise). Trunk-based development commits frequently to main using short branches and feature flags, suited to teams with strong CI/CD and multiple daily deployments (e.g., SaaS/product companies).

Q21. How would you set up Git so backend and frontend teams don't block each other, but still integrate smoothly?

A: Separate feature branches per ticket for each team, merged into a shared `develop` branch through PRs/MRs gated by CI (backend runs `mvn test`, frontend runs `npm test`), with an integration environment auto-deployed from develop for cross-team testing before release.

Q22. What is `git bisect` and when have you used it?

A: A binary-search tool to find which commit introduced a bug by marking known good/bad commits; Git automatically checks out midpoints until the exact breaking commit is found. Useful in real projects to trace regressions across hundreds of commits quickly.

26. Final Summary

Aspect	Before Git	After Git (+ GitHub/GitLab/Bitbucket)
Collaboration	Zip files / email / shared folders	Branches + Pull/Merge Requests
History	None or manual backups	Full history, every change traceable
Code review	Ad-hoc, undocumented	Structured PR/MR review with approvals
Release process	Manual file copy to servers	Tags + automated CI/CD pipelines
Recovery from mistakes	Often impossible	<code>revert</code> , <code>reflog</code> , tags — almost always recoverable
Governance	None	Org roles, branch protection, CODEOWNERS, audit logs

Key takeaway for interviews: Be ready to explain not just *what* a command does, but *why* and *when* a real team uses it — e.g., rebase for clean local history vs merge for shared branches, cherry-pick for targeted hotfixes, revert instead of reset on production branches, and how PR/MR approval gates protect an organization's codebase.